

# Simon Chen

347-322-4133 | [shangminch@gmail.com](mailto:shangminch@gmail.com) | [simon-chen.com](http://simon-chen.com) | [linkedin.com/in/shangmin-chen](https://linkedin.com/in/shangmin-chen) | [github.com/shangmin-chen](https://github.com/shangmin-chen)

## EDUCATION

### Boston University

Boston, MA

*Bachelor of Arts in Computer Science*

*Graduated: Spring 2026*

- Relevant Coursework: Data Structures & Algorithms, Operating Systems, Distributed Systems, Databases, Machine Learning, Data Science.

## EXPERIENCE

### Software Engineering Intern

February 2026 – June 2026

*RESET Standard*

*Shanghai, China*

- Engineered asynchronous data pipelines with RabbitMQ, Redis, and Sidekiq to process long-running ingestion jobs in the background, decoupling external API calls from user-facing requests.
- Doubled the platform's supported environmental data categories (from 2 to 4) by independently implementing end-to-end water and waste data integration, developing Rails backend ingestion pipelines, PostgreSQL schemas, and client-facing React/Vite/Tailwind dashboards.

### Lead Product Engineer

December 2025 – Present

*EZ Esports*

*New York, NY*

- Designed a PostgreSQL schema on Supabase using Drizzle ORM, supporting 500+ matches across 20+ teams, and built RESTful endpoints for player statistics, standings, and historical data.
- Migrated an esports league platform from Google Sites to Next.js with a Supabase backend, replacing manual content updates with live scoring and automated season management.

## PROJECTS

### Persephone | *Python, Cython/C++17, Rust, Modal, FAISS*

May 2026

- Built and operated a live Kalshi BTC prediction-market trading system end to end (market-data ingestion, theoServing, strategy evaluation, execution, and risk), generating \$8K net PnL on \$200K traded volume in its first 4 days.
- Rewrote 9 live hot-path kernels (order-book maintenance, order-book imbalance, trade intensity, event aggregation) as native Cython extensions, cutting Kalshi book maintenance from 21.8ms to 195 $\mu$ s p50 (112x).
- Designed a lock-free SPSC event ring in Cython/C++17 with cache-line-aligned atomic cursors and packed uint64 records, benchmarked at 584ns per enqueue and 703ns per event drained.
- Eliminated duplicate-send and crash-replay risk with a durable-before-wire order layer: intent journaled before dispatch, deterministic client IDs for idempotent retries, and persisted ack state for reconciliation.
- Prevented bad-price quoting with fail-closed gates that block stale, crossed, or untrusted market data before quote generation, computing fused market metrics in 357 $\mu$ s p50 across 8 venues.
- Replaced KDE inference with a GBM theoS layer, cutting model query latency from  $\sim$ 1.5ms to  $\sim$ 20 $\mu$ s per query ( $\sim$ 75x) behind an unchanged live serving interface.
- Cut backtest and parameter-sweep iteration from  $\sim$ 14 hours on laptop CPUs to  $\sim$ 15 minutes by moving tick backtests and sweeps to Modal NVIDIA GPU workers with FAISS GPU search and CuPy-vectorized math.
- Sped up 155K-row neighbor search from 758 $\mu$ s to 154 $\mu$ s p50 by caching FAISS IVF indexes and pinning FAISS threads to avoid OpenMP oversubscription.

### Hermes Letters | *Next.js, TypeScript, Supabase, Drizzle ORM, PostgreSQL*

June 2026

- Ensured consistent letter claims via atomic SQL updates and secured sessions in XSS-resistant httpOnly cookies.
- Derived user connections at query time from kept-letter records using indexed lookups, avoiding a separate relationship table and associated database sync issues.
- Hardened media storage by validating uploads via server-side magic-byte signatures, storing files in a private Supabase bucket, and restricting delivery to 1-hour signed URLs.

## TECHNICAL SKILLS

**Languages:** Python, C++17, Cython, Rust, TypeScript, Java, SQL

**Frameworks & Libraries:** FAISS, CuPy, FastAPI, React, Next.js

**Databases & Infrastructure:** PostgreSQL, Redis, RabbitMQ, Docker, Nginx, Modal, OpenMP